

Contents

SEAS-MFEM Codebase Guide	2
Table of Contents	2
1. Directory Layout	3
2. Execution Flow: One Time Step	4
3. Drivers	5
3.1 seas_driver.cpp (primary, TOML-based)	5
3.2 pseas.cpp (legacy BP2 driver)	7
4. Configuration System	7
4.1 SEASConfig – top-level data struct	7
4.2 SEASConfigParser – TOML file parsing	7
4.3 SEASConfigBridge – config to internal objects	8
5. Domain Operators	8
5.1 Abstract Base: DomainOperator<MeshType>	8
5.2 ElasticityDomainOperator<MeshType> – 3D DG Elasticity	9
5.3 AntiplaneDomainOperator<MeshType> – 2D DG Scalar	11
6. DG Integrators (Weak Form to Code)	11
6.1 The Complete DG Weak Form (IP Method)	11
6.2 IP Combined Integrator	12
6.3 BR2 Integrator (3D Elasticity)	15
6.4 Scalar BR2 (2D Antiplane)	16
7. Fault Module	16
7.1 RateStateFaultOperator<MeshType, SlipComponents>	16
7.2 FaultGeometry<MeshType>	17
7.3 FaultBasis	17
7.4 FaceQuadrature and L2 Projection	18
8. Friction and State Evolution	20
8.1 DieterichRuinaFriction	20
8.2 State Evolution Laws	21
9. Time Integration	22
9.1 PETSc TS (default)	22
9.2 Native DormandPrince RK45 (fallback)	22
10. Constitutive Models	23
10.1 ConstitutiveModel (abstract)	23
10.2 LinearElastic	23
11. Parallelism and MPI	23
11.1 MPI Operations in the ODE Evaluation Flow	24
11.2 MPIContext	24
11.3 FaultScatter	25
12. I/O and Benchmark Output	25
12.1 ProbeOutput	26
12.2 ParallelBP5BenchmarkOutput	26
13. Coordinate Conventions and Sign Rules	26
Coordinate System	26
Mesh Boundary Attributes	26
Sign Rules	26
14. State Vector Layout	27

Antiplane (SlipComponents=1, StatePerNode=2)	27
3D Elasticity (SlipComponents=2, StatePerNode=3)	27
Slip and Traction Vectors	27
15. Appendix: DG-IP Derivation for 3D Elasticity	27
15.1 Strong Form	27
15.2 First-Order System and Weak Form	28
15.3 Flux Form	28
15.4 Eliminating σ (Primal Form)	28
15.5 Summing Over Elements and the Magic Identity	29
15.6 Choosing the IP Numerical Fluxes	29
15.7 Assembling the Primal Form	29
15.8 Structure of the Bilinear Form	30
15.9 Mapping to Code	30

SEAS-MFEM Codebase Guide

Quick-reference guide to the SEAS miniapp source code. Every section links to files, functions, and code snippets so you can navigate directly to the relevant implementation. Equations are provided alongside code to show the math-to-code mapping.

Last updated: 2026-04-12 (v3: full DG-IP derivation, expanded weak form + code mapping, MPI flowchart).

For a rendered version:

```
pandoc CODEBASE_GUIDE.md -o CODEBASE_GUIDE.html --standalone --embed-resources --mathjax
open CODEBASE_GUIDE.html
```

Table of Contents

1. Directory Layout
2. Execution Flow: One Time Step
3. Drivers
4. Configuration System
5. Domain Operators
6. DG Integrators (Weak Form to Code)
7. Fault Module
8. Friction and State Evolution
9. Time Integration
10. Constitutive Models
11. Parallelism and MPI
12. I/O and Benchmark Output
13. Coordinate Conventions and Sign Rules
14. State Vector Layout
15. Appendix: DG-IP Derivation for 3D Elasticity

1. Directory Layout

```
miniapps/seas/
  drivers/
    seas_driver.cpp      # Primary driver (TOML config, PETSc TS default)
    pseas.cpp            # Legacy parallel BP2 driver (hardcoded params)

  config/                # Configuration and parameter files
    seas_config.hpp      # SEASConfig: top-level config data struct
    seas_config_parser.hpp # TOML parser -> SEASConfig
    seas_config_bridge.hpp # SEASConfig -> BP5Params/BoundaryConfig/DomainConfig
    bp5_params.hpp       # BP5 physical parameters + spatial functions a(x2,x3)
    bp2_params.hpp       # BP2 physical parameters + depth functions a(z)
    bp5_mesh_utils.hpp   # Inline mesh creation for quick tests

  domain/                # PDE domain solvers
    domain_operator.hpp  # Abstract base: Solve(), ComputeTraction()
    elasticity_operator.hpp # 3D DG elasticity: IP/BR2, MUMPS/CG, fault slip RHS
    antiplane_operator.hpp # 2D DG Laplace: scalar antiplane shear
    boundary_config.hpp  # BoundaryConfig struct + DirichletFunc factory
    domain_config.hpp    # DomainConfig struct (penalty, basis type, etc.)
    bp2_mesh.hpp         # BP2 graded mesh generator
    seas_boundary_tags.hpp # Fault boundary detection from mesh attributes

  fault/                 # Fault ODE operator and geometry
    rate_state_fault.hpp # RateStateFaultOperator: PreInit/Init/ComputeRHS
    fault_geometry.hpp   # FaultGeometry: per-DOF a, eta, Dc, tau_pre, V_init
    fault_basis.hpp      # FaultBasis: per-face (normal, dip, strike) frames
    fault_nodes.hpp      # Fault DOF node management
    face_quadrature.hpp  # FaceQuadrature: multi-DOF L2 projection at p>=2

  friction/              # Friction laws and state evolution
    friction_law.hpp     # Abstract FrictionLaw interface
    dieterich_ruina.hpp  # DieterichRuinaFriction: Brent solver, psi-space
    state_evolution.hpp  # AgingLaw, SlipLaw, AgingLawPsi, SlipLawPsi

  solver/                # SEAS coupling operator and time steppers
    seas_operator.hpp    # SEASQuasiDynamicOperator: domain+fault coupling
    time_stepper.hpp     # DormandPrinceRK45 (fallback) + AdaptiveTimeStepper

  integrator/            # DG bilinear form integrators
    dg_elasticity_ip_combined_integrator.hpp # IP: stiffness + slip RHS + traction
    dg_elasticity_br2_integrator.hpp         # BR2 lifting operator (3D elasticity)
    dg_elasticity_ip_penalty_integrator.hpp  # Legacy IP penalty-only
    dg_br2_integrator.hpp                    # Scalar BR2 (2D antiplane)

  constitutive/          # Material models
    constitutive_model.hpp # Abstract ConstitutiveModel (Tier 1-3 design)
```

```

linear_elastic.hpp      # LinearElastic: isotropic Hooke's law

io/                     # Output writers
  bp5_parallel_output.hpp # Distributed SCEC fltst probe output
  parallel_benchmark_output.hpp # Distributed probe output (BP2)
  probe_output.hpp       # Single-station ASCII file writer
  checkpoint.hpp         # Checkpoint/restart

common/                # Shared utilities
  seas_types.hpp        # Serial/parallel type aliases
  mpi_context.hpp       # MPIContext: RAII MPI wrapper with reductions
  fault_scatter.hpp     # FaultScatter: MPI point-to-point for ghost DOFs

tests/
  unit/                 # Unit tests (make test)
  parallel/             # MPI parallel tests (mpirun -np N)
  verification/        # Long-running benchmark verification

```

2. Execution Flow: One Time Step

The default time stepper is **PETSc TS** (adaptive RK45, configured via `use_petsc_ts = true` in the TOML config). A native `DormandPrinceRK45` implementation is available as a fallback when PETSc is not compiled in. Both steppers call the same ODE right-hand-side function `Mult(state, rate)` which performs one complete physics evaluation:

PETSc TS / DormandPrinceRK45

```

|
+-- seas_op.Mult(state, rate) [solver/seas_operator.hpp:318]
|
|   Step 1: Extract slip from ODE state vector
+-- fault_>GetSlip(state, slip_) [fault/rate_state_fault.hpp]
|   Unpacks [s_dip_0, s_strike_0, psi_0, ...] -> slip[2*N]
|
|   Step 2: Ghost exchange + solve bulk elasticity
+-- domain_>ExpandOwnedToLocalFault(slip_, local_slip_, 2)
|   MPI point-to-point: owned fault DOFs -> shared ghost DOFs
+-- domain_>Solve(t, local_slip_, u_gf_)
|   Assembles RHS from slip + Dirichlet BCs, solves  $K*u = b$ 
|   (K is cached after first call; RHS rebuilt every call)
|
|   Step 3: Recover traction on fault from displacement
+-- domain_>ComputeTraction(u_gf_, local_slip_, local_traction_)
|   Evaluates DG numerical flux traction at fault faces
+-- domain_>RestrictToOwnedFault(local_traction_, traction_, 2)
|   Extracts owned DOFs from full local fault view
|
|   Step 4: Friction solve -> slip rate + state rate

```

```

+--- fault_-->ComputeRHS(traction_, state, rate)
    For each fault DOF i:
        tau = tau_pre[i] + traction[i]  (total stress)
        V = SolveSlipRateVectorPsi(tau, psi, sigma_n, eta, a)
        rate[i] = [V_dip, V_strike, dpsi/dt]

```

Data Flow Diagram (shows how data transforms between steps):

```

state = [slip_dip, slip_strike, psi] x N_fault
|
Step 1: | GetSlip() extracts slip components
        v
        slip (fault-local, dip+strike per DOF)
        |
Step 2a: | ExpandOwned: MPI ghost exchange (FaultScatter)
        | EmbedSlipQP: (dip,strike) -> (x,y,z) via tangent frames
        v
        slip (global 3D coordinates, full local fault view)
        |
Step 2b: | Solve(): assemble RHS, solve K*u = b
        v
        u (displacement field, entire domain)
        |
Step 3:  | ComputeTraction: T = {sigma.n} - penalty*(jump - slip)
        | RestrictToOwned: keep only owned DOFs
        | Project: (x,y,z) -> (dip,strike) via tangent frames
        v
        traction (fault-local, dip+strike per owned DOF)
        |
Step 4:  | ComputeRHS: friction solve per DOF
        |   tau_total = tau_pre + traction
        |   V = solve: |tau| = sigma_n * f(|V|, psi) + eta * |V|
        |   dpsi/dt = aging_law(|V|, psi, Dc)
        v
        rate = [V_dip, V_strike, dpsi/dt] x N_fault

```

3. Drivers

3.1 seas_driver.cpp (primary, TOML-based)

File: drivers/seas_driver.cpp

Reads all configuration from a TOML file. Supports **PETSc TS** (default) and native Dormand-Prince RK45 (fallback). The pipeline has 9 stages; Stages 1-6 set up the simulation, Stage 7 configures the time stepper, Stage 8 runs the time loop, Stage 9 writes final output.

```

// Stage 1: Parse TOML config [line 227]
SEASConfig config = SEASConfigParser::ParseFile(config_file);
SEASConfigParser::ApplyCLIOverrides(config, overrides);

```

```

SEASConfigParser::Validate(config);

// Stage 2: Load mesh (from file or inline generator) [line 277]
ParMesh pmesh(mpi.GetComm(), *serial_mesh);

// Stage 3: Create domain operator with constitutive model [line 327]
// Constructs FE space, detects fault faces, builds stiffness solver
LinearElastic material(params.lambda(), params.mu());
ElasticityDomainOperator<ParMesh> domain(
    pmesh, config.mesh.order, material,
    params.Vp, params.Wf, params.lf,
    bdr_config, dg_method, solver_type, domain_config);

// Stage 4: Create fault components [line 345]
// Precomputes per-DOF a(x2,x3), Dc(x2,x3), tau_pre, V_init
FaultGeometry<ParMesh> fault_geom(domain, params, &mpi);
DieterichRuinaFriction friction(fc);
AgingLawPsi aging(params.b, params.V0, params.f0);
RateStateFaultOperator<ParMesh, 2> fault_op(
    &fault_geom, &friction, &aging, params, &mpi);

// Stage 5: Wire domain + fault into SEAS operator, initialize [line 361]
// 4-phase init: PreInit -> domain solve -> Init -> verify equilibrium
PBP5SEASOp seas_op(&domain, &fault_op, &mpi);
seas_op.SetInitialCondition(state);

// Stage 6: Set up I/O (probe stations, global output) [line 377]
ParallelBP5BenchmarkOutput bench_out(...);

// Stage 7: Configure PETSc TS (default) or DormandPrince [line 423]
// PETSc path: creates PetscODESolver, sets TSAdaptBasic,
// registers TSMonitor callback for I/O during TSSolve.
// Native path: configures DormandPrinceRK45 with tolerances.
if (use_petsc_ts) {
    petsc_ode = std::make_unique<PetscODESolver>(mpi.GetComm(), "");
    petsc_ode->Init(seas_op, PetscODESolver::ODE_SOLVER_GENERAL);
    TSMonitorSet(ts, driver_ts_monitor_callback, &petsc_mon_ctx, nullptr);
}

// Stage 8: Time loop [line 527]
// PETSc: single call to petsc_ode->Run(state, t, dt, t_final)
// Native: while-loop calling ode_solver.Step(seas_op, state, t, dt)
// Both paths do earthquake detection, adaptive I/O, checkpointing.

// Stage 9: Write final state and summary [line 661]
bench_out.ForceWrite(t, state, fault_op, seas_op.GetTraction(), V_max);

```

3.2 pseas.cpp (legacy BP2 driver)

File: pseas.cpp

Hardcoded BP2 parameters, no TOML config. Uses the antiplane (scalar) path:

```
AntiplaneDomainOperator<ParMesh> domain(pmesh, order, params.mu(), params.Vp, params.Wf, DGMethod::BR2);
RateStateFaultOperator<ParMesh> fault_op(&fault_geom, &friction, &aging, params, &mpi); // SlipComponent
SEASQuasiDynamicOperator<ParMesh> seas_op(&domain, &fault_op, &mpi);
```

The antiplane problem solves $\nabla^2 u = 0$ (scalar) instead of 3D vector elasticity. The domain operator, fault operator, and SEAS coupling operator are the same classes with different template parameters.

4. Configuration System

Follows Stage 1 of the driver. Produces the config objects consumed by Stages 2-7.

4.1 SEASConfig – top-level data struct

File: config/seas_config.hpp

Pure data struct with nested sections. No factory methods, no logic:

SEASConfig struct (config/seas_config.hpp):

```
struct SEASConfig {
    std::string benchmark; // "bp5", "bp1", "bp2", or ""
    MeshConfig mesh; // file, scale, order
    MaterialConfig material; // density, cs, nu -> mu(), lambda()
    FrictionConfig friction; // V0, f0, b, L0, L_nuc, a0, amax, sigma_n
    LoadingConfig loading; // Vp, V_init, V_nuc, delta_tau_factor
    FaultGeomConfig fault_geom; // Wf, lf, hs, ht, H, l_vw, w_nuc
    BoundaryTomlConfig boundary; // dirichlet_attrs, natural_attrs, fault_attr
    SolverConfig solver; // dg_method, solver_type, penalty_factor
    TimeConfig time; // t_final, atol, rtol, use_petsc_ts
    OutputConfig output; // output_dir, output_prefix
};
```

MaterialConfig derives Lamé parameters from wave speed and Poisson's ratio:

Lamé parameter derivation (MaterialConfig, lines 41-42):

```
// mu = rho * cs^2 (shear modulus) [line 41]
real_t mu() const { return density * cs * cs; }
// lambda = 2*nu*mu / (1 - 2*nu) (first Lamé parameter) [line 42]
real_t lambda() const { return 2.0 * nu * mu() / (1.0 - 2.0 * nu); }
```

4.2 SEASConfigParser – TOML file parsing

File: config/seas_config_parser.hpp. Requires SEAS_USE_TOML build flag.

- ParseFile(filepath) [line 42]: reads TOML, returns SEASConfig

- `ApplyBP5Defaults(config)` [line 308]: if `benchmark = "bp5"`, fills all fields from `BP5Params` defaults
- `ApplyCLIOverrides(config, overrides)` [line 87]: applies "key=value" overrides (e.g., "`time.t_final=1e10`")
- `Validate(config)` [line 129]: checks `mesh.file` non-empty, `nu` in (0, 0.5), `dg_method` is "IP" or "BR2", etc.

4.3 SEASConfigBridge – config to internal objects

File: `config/seas_config_bridge.hpp`

Converts `SEASConfig` fields to the internal parameter/config objects used by operators:

- `BuildBP5Params(config)` [line 28]: copies material/friction/loading fields to `BP5Params`
- `ParseDGMethod(s)` [line 61]: "IP" -> `DGMethod::IP`, "BR2" -> `DGMethod::BR2`
- `ParseSolverType(s)` [line 70]: "mumps" -> `SolverType::MUMPS`, "cg" -> `SolverType::CG_AMG`, etc.
- `BuildBoundaryConfig(bdr, Vp)` [line 83]: creates `BoundaryConfig` with Dirichlet function
- `BuildDomainConfig(solver)` [line 95]: creates `DomainConfig` with penalty/basis/tolerance settings

5. Domain Operators

Consumed by Step 2 (Solve) and Step 3 (ComputeTraction) of each ODE evaluation.

5.1 Abstract Base: `DomainOperator<MeshType>`

File: `domain/domain_operator.hpp`

Template on `MeshType` (`Mesh` for serial, `ParMesh` for parallel). This base class defines the interface that the SEAS coupling operator (`SEASQuasiDynamicOperator`) uses to interact with the domain PDE solver. Two concrete implementations exist: `ElasticityDomainOperator` (3D vector) and `AntiplaneDomainOperator` (2D scalar).

Method	Purpose
<code>Solve(time, slip_bc, displacement)</code>	Given current time and fault slip values, solve the domain equilibrium equation and return the displacement field. Slip is imposed as a jump condition on interior fault faces; Dirichlet BCs provide far-field loading.
<code>ComputeTraction(displacement, slip_bc, traction, normal_traction)</code>	Given the displacement solution and prescribed slip, evaluate the DG numerical flux traction on all fault faces. Returns shear traction (dip + strike components) and optionally the normal stress component.
<code>GetNumFaultDOFs()</code>	Return total number of fault DOFs on this rank (includes shared partition-boundary ghosts in parallel).

Method	Purpose
<code>GetNumOwnedFaultDOFs()</code>	Return number of uniquely-owned fault DOFs on this rank. In serial, equals <code>GetNumFaultDOFs()</code> . In parallel, excludes ghost DOFs at partition boundaries so each DOF is counted exactly once across all ranks.
<code>NumSlipComponents()</code>	Return 1 for antiplane (scalar slip) or 2 for 3D elasticity (dip + strike slip components).
<code>RestrictToOwnedFault(local, owned, comps)</code>	Extract the owned-only subset from a full local fault vector. In serial, this is a copy. In parallel, it removes ghost DOFs. Used after <code>ComputeTraction()</code> to get traction only at owned DOFs.
<code>ExpandOwnedToLocalFault(owned, local, comps)</code>	Broadcast owned fault data to the full local view (including ghost DOFs). In parallel, performs MPI point-to-point communication via <code>FaultScatter</code> so that shared partition-boundary faces have consistent slip values. Used before <code>Solve()</code> .
<code>GetFaultBasis()</code>	Return pointer to the per-face coordinate frame (<code>FaultBasis</code>). Used to convert between fault-local (dip, strike) and global (x, y, z) coordinates. Returns <code>nullptr</code> for antiplane (no rotation needed).

5.2 ElasticityDomainOperator<MeshType> – 3D DG Elasticity

File: domain/elasticity_operator.hpp (584 lines + .inl files)

This is the **general-purpose 3D DG elasticity** domain operator. It solves

$$\nabla \cdot \sigma(\mathbf{u}) = \mathbf{0}$$

where $\sigma = \lambda \text{tr}(\varepsilon) \mathbf{I} + 2\mu \varepsilon$ using Discontinuous Galerkin methods (IP or BR2) with fault slip as an interior jump BC. While current benchmarks use BP5 parameters, the operator accepts any `ConstitutiveModel` and `BoundaryConfig` and is not tied to any specific benchmark problem.

Constructor (line 104):

ElasticityDomainOperator constructor signature:

```
ElasticityDomainOperator(MeshType &mesh, int order,
    const ConstitutiveModel &model, // material law (non-owning)
    real_t Vp,                      // plate rate [m/s]
    real_t Wf,                      // fault depth [m]
    real_t lf,                      // fault length [m]
    const BoundaryConfig &bdr_config, // which attrs are Dirichlet/Natural/Fault
    DGMethd method,                 // IP or BR2
    SolverType solver_type,          // MUMPS, CG_AMG, etc.
    const DomainConfig &config);    // penalty factor, basis type, etc.
```

InitOperator() (line 341) – setup sequence called by constructor:

Initialization pipeline:

```
void InitOperator() {
    epsilon_ = -1.0;           // SIPG sign (symmetric interior penalty)
    SetupFESpace();           // Create DG_FECollection + vector FE space (vdim=3)
    SetupBoundaryMarkers();    // Classify mesh faces as Dirichlet/Natural/Fault from attrs
    SetupFaultInfo();         // Identify fault interior+shared faces, build FaultBasis frames,
                                // create FaceQuadrature for multi-DOF at p>=2
    RunStartupFaceAudit();     // Verify face classification is consistent across ranks
    SetupSolver();             // Create MUMPS/CG/SuperLU solver instance
}
```

Solve(time, slip_bc, displacement) – assembles and solves the linear system. Called by SEASQuasiDynamicOperator::Mult() at every RK stage (see Section 2, Step 2b):

1. **AssembleStiffness()** [assembly.inl:9] – build the stiffness matrix K once (cached):
 - Volume term: $\int_{\Omega} \sigma(\mathbf{u}) : \varepsilon(\mathbf{v}) dV$
 - DG face terms: consistency + symmetry + penalty (see Section 6)
2. **AssembleSlipContribution(rhs, slip_bc)** [assembly.inl:420] – build $\mathbf{b}_{\text{slip}}$:
 - For each fault face: embed slip from fault-local to global 3D, then compute the DG face integral using the same integrator as K (ensures consistency)
3. **AssembleDirichletLoading(rhs, time)** [assembly.inl:1040] – build $\mathbf{b}_{\text{dir}}$:
 - Evaluate $\mathbf{u}_D(\mathbf{x}, t)$ at boundary quadrature points, integrate DG Dirichlet terms
4. **solver_→Mult(B_, X_)** – solve $K\mathbf{u} = \mathbf{b}_{\text{slip}} + \mathbf{b}_{\text{dir}}$

ComputeTraction(displacement, slip_bc, traction) – recovers numerical flux traction on fault faces. Called by SEASQuasiDynamicOperator::Mult() at Step 3. The traction formula and its code implementation are described in Section 6.

Boundary condition handling (BoundaryConfig, boundary_config.hpp):

BoundaryConfig data struct:

```
struct BoundaryConfig {
    std::set<int> dirichlet_attrs; // e.g., {5} for far-field vertical faces
    std::set<int> natural_attrs;  // e.g., {1} for top/bottom free surfaces
    int fault_attr = 3;           // interior faces tagged with this attribute
    DirichletFunc default_dirichlet_func; // (x, t) → prescribed displacement
};
```

The Dirichlet function for the SCEC BP5 benchmark problem applies far-field plate-rate loading. At distance $|y| > 1000$ m from the fault, $u_x = \pm V_p t/2$:

BP5 Dirichlet function (boundary_config.hpp):

```
inline DirichletFunc MakeBP5DirichletFunc(real_t Vp) {
    return [Vp](const Vector &x, real_t t, Vector &u) {
        u.SetSize(3); u = 0.0;
        real_t y = x(1); // fault-normal coordinate
        real_t Vh = Vp * t;
        if (y > 1000.0) { Vh *= 0.5; } // +Y side: right-lateral
    }
```

```

    else if (y < -1000.0) { Vh *= -0.5; } // -Y side: left-lateral
    u(0) = Vh; // along-strike displacement
};
}

```

5.3 AntiplaneDomainOperator<MeshType> – 2D DG Scalar

File: domain/antiplane_operator.hpp

Solves the 2D scalar Laplace equation $\mu \nabla^2 u = 0$ with slip jump on the fault at $x = 0$. Uses DG_FECollection with `vdim = 1`. Same interface as `ElasticityDomainOperator` but `NumSlipComponents() = 1` and `NumComponents() = 1`.

6. DG Integrators (Weak Form to Code)

These integrators are the numerical core of the domain operator. They implement the DG weak form equations as MFEM bilinear/linear form integrators. Understanding this section is essential for modifying the stiffness assembly, slip RHS, or traction recovery. For the full derivation from strong form to this DG weak form, see Section 15.

6.1 The Complete DG Weak Form (IP Method)

Notation. Let Ω be the domain discretized into elements $\{K\}$. The boundary $\partial\Omega$ is partitioned into Dirichlet Γ^D , Neumann Γ^N , and fault Γ^F faces. Interior non-fault faces are Γ^0 . For two elements K^-, K^+ sharing face e , with outward normal $\mathbf{n}$ pointing from K^- to K^+ :

- **Average:** $\{\{\mathbf{q}\}\} = \frac{1}{2}(\mathbf{q}^- + \mathbf{q}^+)$
- **Jump:** $\llbracket \mathbf{u} \rrbracket = \mathbf{u}^- - \mathbf{u}^+$
- **Stress:** $\sigma(\mathbf{u}) = \mathbb{C} : \varepsilon(\mathbf{u})$, where $C_{ijkl} = \lambda \delta_{ij} \delta_{kl} + \mu(\delta_{ik} \delta_{jl} + \delta_{il} \delta_{jk})$

The **complete IP bilinear form** $a(\mathbf{u}, \mathbf{v})$ is (derived in Section 15):

$$\begin{aligned}
a(\mathbf{u}, \mathbf{v}) = & \underbrace{\sum_K \int_K \nabla \mathbf{v} : \mathbb{C} : \nabla \mathbf{u} \, dx}_{\text{(I) volume stiffness}} \\
& - \underbrace{\sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e (\llbracket \mathbf{u} \rrbracket \cdot \{\{\mathbb{C} : \nabla \mathbf{v}\}\} \mathbf{n} + \llbracket \mathbf{v} \rrbracket \cdot \{\{\mathbb{C} : \nabla \mathbf{u}\}\} \mathbf{n}) \, ds}_{\text{(II-a, III-a) interior/fault consistency + symmetry}} \\
& + \underbrace{\sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e \frac{\eta_e}{h_e} \llbracket \mathbf{u} \rrbracket \cdot \llbracket \mathbf{v} \rrbracket \, ds}_{\text{(III-b) interior/fault penalty}} \\
& - \underbrace{\sum_{e \in \Gamma^D} \int_e (\mathbf{u} \cdot (\mathbb{C} : \nabla \mathbf{v}) \mathbf{n} + \mathbf{v} \cdot (\mathbb{C} : \nabla \mathbf{u}) \mathbf{n}) \, ds}_{\text{(II-a, III-a) Dirichlet consistency + symmetry}}
\end{aligned}$$

$$+ \underbrace{\sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} \mathbf{u} \cdot \mathbf{v} \, ds}_{\text{(III-b) Dirichlet penalty}}$$

The **linear form** $L(\mathbf{v})$ collects boundary data and fault slip δ :

$$\begin{aligned} L(\mathbf{v}) = & - \sum_{e \in \Gamma^D} \int_e \mathbf{g}^D \cdot (\mathbb{C} : \nabla \mathbf{v}) \mathbf{n} \, ds + \sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} \mathbf{g}^D \cdot \mathbf{v} \, ds \\ & - \sum_{e \in \Gamma^F} \int_e \delta \cdot \{ \{ \mathbb{C} : \nabla \mathbf{v} \} \} \mathbf{n} \, ds + \sum_{e \in \Gamma^F} \int_e \frac{\eta_e}{h_e} \delta \cdot \llbracket \mathbf{v} \rrbracket \, ds \end{aligned}$$

where $\mathbf{g}^D$ is the prescribed Dirichlet displacement and δ is the prescribed fault slip. The problem is: **find** $\mathbf{u} \in V_h$ **such that** $a(\mathbf{u}, \mathbf{v}) = L(\mathbf{v})$ **for all** $\mathbf{v} \in V_h$.

The **traction operator** $[\sigma(\phi_k \mathbf{e}_i) \cdot \mathbf{n}]_u$ contracts the stress of a basis function ϕ_k in direction i with normal $\mathbf{n}$ in direction u :

$$[\sigma(\phi_k \mathbf{e}_i) \cdot \mathbf{n}]_u = \lambda \frac{\partial \phi_k}{\partial x_i} n_u + \mu \left(\delta_{iu} (\nabla \phi_k \cdot \mathbf{n}) + \frac{\partial \phi_k}{\partial x_u} n_i \right)$$

This couples all displacement components through λ and μ . Both stiffness assembly and traction recovery use this same operator (ensuring K-b consistency).

6.2 IP Combined Integrator

File: `integrator/dg_elasticity_ip_combined_integrator.hpp`

A single integrator that handles stiffness matrix assembly, slip RHS contribution, and traction recovery for the IP method. This design ensures that all three operations use the same traction operator and quadrature rule.

6.2.1 IP Penalty Parameter **ComputePenalty** (lines 841-861). For polynomial order p in spatial dimension $d = 3$:

$$c_{N,1} = \frac{p(p+d-1)}{d}$$

$$c_0 = 2\mu, \quad c_1 = d\lambda + 2\mu, \quad \text{ratio} = \frac{c_1^2}{c_0}$$

$$p_k = (d+1) \cdot c_{N,1} \cdot \frac{d \cdot |\mathbf{n}_q|}{|\det J_k|} \cdot \text{ratio}, \quad k \in \{1, 2\}$$

$$\alpha = \text{penalty_factor} \cdot \frac{p_1 + p_2}{4} \quad (\text{interior}), \quad \alpha = \text{penalty_factor} \cdot p_1 \quad (\text{boundary})$$

The ratio term couples λ and μ through the elasticity tensor. A scalar-only penalty (using just μ) is insufficient for 3D stability.

Penalty computation (dg_elasticity_ip_combined_integrator.hpp, lines 847-860):

```
int p = std::max(fe1.GetOrder(), is_interior ? fe2.GetOrder() : fe1.GetOrder());
real_t c_N_1 = p * (p + dim_ - 1.0) / dim_;           // SIP stability constant
real_t c0 = 2.0 * mu_val;                             // = 2*mu
real_t c1 = dim_ * lam + 2.0 * mu_val;                // = d*lambda + 2*mu
real_t ratio = c1 * c1 / c0;                          // elasticity tensor coupling

real_t p0 = (dim_ + 1) * c_N_1 * (real_t(dim_) * nl_q / detJ1) * ratio; // element 1
if (!is_interior) { return penalty_factor_ * p0; }
real_t p1 = (dim_ + 1) * c_N_1 * (real_t(dim_) * nl_q / detJ2) * ratio; // element 2
return penalty_factor_ * (p0 + p1) / 4.0;             // averaged for interior faces
```

Here $nl_q = |\mathbf{n}_q|$ is the unnormalized face normal length from `CalcOrtho()`, and `detJ1`, `detJ2` are the Jacobian determinants of elements 1 and 2.

6.2.2 Stiffness Matrix Assembly **AssembleFaceMatrix** (lines 57-194) assembles the face contribution to K . For each quadrature point q on the face, three coefficients control the three DG terms:

Block	c0 (consistency)	c1 (symmetry)	c2 (penalty)
(K^-, K^-)	-0.5	$+0.5\varepsilon$	$+\alpha$
(K^-, K^+)	-0.5	-0.5ε	$-\alpha$
(K^+, K^-)	+0.5	$+0.5\varepsilon$	$-\alpha$
(K^+, K^+)	+0.5	-0.5ε	$+\alpha$

where $\varepsilon = -1$ for SIPG. For boundary faces, only the (K^-, K^-) block exists with $c0 = -1$, $c1 = \text{epsilon}$, $c2 = +\text{penalty}$.

Each `AssembleFaceBlock` call computes, for test DOF (k, p) and trial DOF (l, u) :

$$K_{kp,lu} += w_q \left[c_0 \cdot \frac{[\sigma(\phi_l \mathbf{e}_u) \cdot \mathbf{n}]_p}{\det J_{\text{trial}}} \cdot \phi_k + c_1 \cdot \frac{[\sigma(\phi_k \mathbf{e}_p) \cdot \mathbf{n}]_u}{\det J_{\text{test}}} \cdot \phi_l + c_2 \cdot |\mathbf{n}_q| \cdot \phi_k \cdot \phi_l \cdot \delta_{pu} \right]$$

6.2.3 Slip RHS Assembly **AssembleSlipFaceRHS** (lines 210-327) computes the linear form contribution from prescribed fault slip δ . From the complete weak form, the fault slip terms in $L(\mathbf{v})$ are:

$$L_{\text{slip}}(\mathbf{v}) = - \sum_{e \in \Gamma^F} \int_e \delta \cdot \{ \{ \mathbb{C} : \nabla \mathbf{v} \} \} \mathbf{n} ds + \sum_{e \in \Gamma^F} \int_e \frac{\eta_e}{h_e} \delta \cdot [\mathbf{v}] ds$$

Discretizing with basis functions ϕ_k and quadrature weight w_q , the element vector entry for test DOF (k, p) on element 1 is:

$$b_1[k, p] = \sum_q w_q \left[\underbrace{\frac{\varepsilon}{2} \cdot \frac{[\sigma(\phi_k \mathbf{e}_p) \cdot \mathbf{n}]_u \cdot \delta_u}{\det J_1}}_{\text{symmetry term}} + \underbrace{\alpha \cdot |\mathbf{n}_q| \cdot \phi_k \cdot \delta_p}_{\text{penalty term}} \right]$$

For element 2, the penalty term has opposite sign: $-\alpha \cdot |\mathbf{n}_q| \cdot \phi_k \cdot \delta_p$. The symmetry coefficient $c_1 = \varepsilon/2$ is the same for both elements.

Slip RHS inner loop (dg_elasticity_ip_combined_integrator.hpp, lines 282-303):

```
// --- Element 1 RHS (side 0) ---
for (int k = 0; k < ndof1; k++)
{
    // Traction operator: [sigma(phi_k * e_p) . n]_u contracted with slip f_q
    real_t grad_dot_n = 0.0; // nabla(phi_k) . n
    real_t grad_dot_f = 0.0; // nabla(phi_k) . f_q
    for (int d = 0; d < dim_; d++)
    {
        grad_dot_n += dshape1_adj(k, d) * nor(d); // dshape1_adj = J^{-T} * dphi/dxi
        grad_dot_f += dshape1_adj(k, d) * f_q[d]; // f_q = slip at quad point
    }

    for (int p = 0; p < dim_; p++)
    {
        // trac_test = lambda * dphi/dx_p * (f.n) + mu * ((dphi.n)*f[p] + (dphi.f)*n[p])
        real_t trac_test = lam1 * dshape1_adj(k, p) * f_dot_n
            + mu1 * (grad_dot_n * f_q[p] + grad_dot_f * nor(p));

        int idx = p * ndof1 + k;
        elvec1(idx) += c1 * trac_test * w_q / detJ1; // symmetry term
        elvec1(idx) += penalty * w_q * nl_q * shape1(k) * f_q[p]; // penalty term (+)
    }
}
```

In this code: $dshape1_adj = \text{adjugate-transformed shape derivatives } \det(J) \cdot J^{-T} \nabla_\xi \phi_k$, $nor = \text{unnormalized face normal from CalcOrtho()}$, $nl_q = |\mathbf{n}_q|$, $f_q[d] = \text{prescribed slip at quadrature point in global 3D coordinates}$, $f_dot_n = \delta \cdot \mathbf{n}$, $c1 = \varepsilon/2 = -0.5$.

6.2.4 Traction Recovery `ComputeTractionAtQuadPoints` (lines 434-554) recovers the numerical flux traction at fault quadrature points. The recovered traction is the stress that the DG solution applies to the fault surface:

$$\mathbf{T}(x_q) = \underbrace{\frac{1}{2} (\sigma^- + \sigma^+) \cdot \hat{\mathbf{n}}}_{\text{stress average}} - \underbrace{\alpha ([\mathbf{u}] - \delta)}_{\text{penalty correction}}$$

where $\hat{\mathbf{n}} = \mathbf{n}/|\mathbf{n}|$ is the unit normal, $\sigma^\pm = \mathbb{C} : \varepsilon(\mathbf{u}^\pm)$ are the stresses on each side, and δ is the prescribed fault slip.

Traction recovery inner loop (dg_elasticity_ip_combined_integrator.hpp, lines 513-551):

```
// --- Stress-average traction ---
// Compute symmetric strain and stress on each side, average, dot with n_hat
real_t tr1 = grad1(0,0) + grad1(1,1) + grad1(2,2); // tr(epsilon^-)
real_t tr2 = grad2(0,0) + grad2(1,1) + grad2(2,2); // tr(epsilon^+)
for (int p = 0; p < dim_; p++)
{
    real_t T_p = 0.0;
    for (int j = 0; j < dim_; j++)
    {
        real_t eps1 = 0.5*(grad1(p,j) + grad1(j,p)); // eps^-_{pj}
        real_t eps2 = 0.5*(grad2(p,j) + grad2(j,p)); // eps^+_{pj}
        real_t sig1 = (p==j ? lam1t*tr1 : 0.0) + 2.0*mu1t*eps1; // sigma^-_{pj}
        real_t sig2 = (p==j ? lam2t*tr2 : 0.0) + 2.0*mu2t*eps2; // sigma^+_{pj}
        T_p += 0.5 * (sig1 + sig2) * n_hat(j); // {sigma}_{pj} * n_hat_j
    }
    traction_q(p * nq + q) = T_p;
}

// --- Penalty correction ---
for (int c = 0; c < dim_; c++)
{
    real_t u1q = 0.0, u2q = 0.0;
    for (int k = 0; k < ndof1; k++) u1q += shape1(k) * u1_dofs(c * ndof1 + k);
    for (int k = 0; k < ndof2; k++) u2q += shape2(k) * u2_dofs(c * ndof2 + k);
    real_t f_q = slip_3d(c * nq + q); // prescribed slip component c
    traction_q(c * nq + q) += (-penalty) * (u1q - u2q - f_q); // -alpha*(jump - slip)
}
```

In this code: $\text{grad1}(p,j) = \partial u_p^- / \partial x_j$ (displacement gradient on element 1), $n_hat(j)$ = unit normal, $\text{shape1}(k)$ = basis function value at quad point, $u1_dofs(c*ndof1+k)$ = displacement DOF for component c , DOF k on element 1.

After recovery, the traction at quadrature points is **L2-projected** onto fault DOFs and **rotated** to fault-local coordinates (dip, strike) – see Section 7.4.

6.3 BR2 Integrator (3D Elasticity)

File: integrator/dg_elasticity_br2_integrator.hpp

The Bassi-Rebay 2 method replaces the IP penalty with a **lifting operator** that reconstructs the gradient correction from the face jump. The penalty becomes:

$$\alpha_{\text{BR2}} \int_{\omega_e} r_e(\llbracket \mathbf{u} \rrbracket) \cdot r_e(\llbracket \mathbf{v} \rrbracket) dx$$

where r_e is the local lifting operator defined by $\int_{\omega_e} r_e(\varphi) \cdot \tau dx = - \int_e \varphi \cdot \{\{\tau\}\} ds$ for all τ , and $\omega_e = K^- \cup K^+$ is the patch of elements sharing face e .

The stabilization parameter is $\eta_e = d + 1$ for tetrahedra ($= 4$) or $2d$ for hexahedra. The lifting is computed via element mass matrix inverses:

BR2 lifting computation (assembly.inl, lines 570-605):

```
// Face integral: face_int[u*dim+s, m] = sum_q shape[m,q] * delta_u[u] * n[s,q] * w[q]
// Lift via inverse mass matrix: f_lifted = 0.5 * Minv * face_int
MultABt(face_int1, Minv1, f_lifted1);
f_lifted1 *= 0.5;
```

6.4 Scalar BR2 (2D Antiplane)

File: integrator/dg_br2_integrator.hpp

Simplified scalar version of BR2 for the 2D Laplace problem $\mu \nabla^2 u = 0$. Same structure as 3D but without the elasticity tensor coupling (no `test_normal`). Penalty: $\eta_e = d + 1 = 3$ for 2D.

7. Fault Module

Consumed by Steps 1 and 4 of each ODE evaluation (see Section 2). The fault module manages the ODE state variables and interfaces with the domain operator (Section 5) and friction solver (Section 8).

7.1 RateStateFaultOperator<MeshType, SlipComponents>

File: fault/rate_state_fault.hpp

Template parameter `SlipComponents`: 1 for antiplane, 2 for 3D elasticity. State per node: `StatePerNode = SlipComponents + 1` (slip components + one state variable).

PreInit(state) [line 188] – called before the first domain solve. Sets initial slip to zero and psi to steady-state placeholder:

PreInit loop (line 188):

```
for (int i = 0; i < num_nodes_; i++) {
    for (int c = 0; c < SlipComponents; c++)
        state(i * StatePerNode + c) = 0.0; // slip = 0
    // psi_ss = f0 + b*ln(V0/V_init) (steady-state for initial slip rate)
    state(i * StatePerNode + PsiIndex) =
        evolution_>SteadyState(V_abs_init, Dc);
}
```

Init(traction, state) [line 239] – called after first domain solve with zero slip. Computes initial psi from stress equilibrium: given total stress τ_0 and initial slip rate V_{init} , find ψ_0 such that $|\tau_0| = \sigma_n f(V_{\text{init}}, \psi_0) + \eta V_{\text{init}}$:

Init: psi from stress equilibrium (line 239):

```
// Total stress = pre-stress + elastic traction from zero-slip solve
real_t tau_vec[2] = {tau_pre_(2*i) + traction(2*i),
                    tau_pre_(2*i+1) + traction(2*i+1)};
```



```

real_t tau_abs = sqrt(tau_vec[0]*tau_vec[0] + tau_vec[1]*tau_vec[1]);
// Solve for psi that satisfies stress equilibrium
real_t psi0 = dr_friction_>InitialStatePsi(tau_abs, V_abs_init, sigma_n, eta, a);
state(i * StatePerNode + PsiIndex) = psi0;

```

ComputeRHS(traction, state, rate, normal_traction) [line 354] – the main ODE RHS. Called at Step 4 of each ODE evaluation. For each fault DOF i :

1. Compute total stress: $\tau_i = \tau_{\text{pre},i} + \tau_{\text{elastic},i}$
2. Compute effective normal stress: $\sigma_{n,\text{eff}} = \sigma_{n,\text{preset}} + T_{n,\text{elastic}}$
3. Solve friction equation for slip rate: $|\tau| = \sigma_n f(|V|, \psi) + \eta |V|$
4. Decompose slip rate: $\mathbf{V} = -(|V|/|\tau|)\tau$ (anti-parallel to stress)
5. Compute state evolution: $\dot{\psi} = (bV_0/D_c)[\exp((f_0 - \psi)/b) - |V|/V_0]$

ComputeRHS per-DOF loop (line 354):

```

real_t psi = state(i * StatePerNode + PsiIndex);
real_t tau_vec[2] = {tau_pre_(2*i) + traction(2*i),
                    tau_pre_(2*i+1) + traction(2*i+1)};
real_t sigma_n_eff = sigma_n_bp5_ + (*normal_traction)(i);

real_t V_vec[2];
dr_friction_>SolveSlipRateVectorPsi(tau_vec, psi, sigma_n_eff, eta, a, V_vec);
real_t V_abs = sqrt(V_vec[0]*V_vec[0] + V_vec[1]*V_vec[1]);

rate(i * StatePerNode + 0) = V_vec[0]; // d(slip_dip)/dt
rate(i * StatePerNode + 1) = V_vec[1]; // d(slip_strike)/dt
rate(i * StatePerNode + 2) = evolution_>Rate(V_abs, psi, Dc); // dps/dt

```

7.2 FaultGeometry<MeshType>

File: fault/fault_geometry.hpp

Created at Stage 4 of the driver. Consumed by RateStateFaultOperator throughout the simulation.

Precomputes per-DOF spatially varying parameters from the domain operator's fault coordinates.

ComputeBP5Params() [line 703] – evaluates parameter functions at each DOF's physical coordinates (x_2, x_3) on the fault surface:

Per-DOF parameter evaluation (line 703):

```

for (int i = 0; i < num_fault_dofs_; i++) {
    real_t x2 = coords_x2_(i), x3 = coords_x3_(i);
    a_values_(i) = bp5_params_.a_of_x2_x3(x2, x3); // direct effect parameter a
    dc_values_(i) = bp5_params_.Dc_of_x2_x3(x2, x3); // critical slip distance [m]
    bp5_params_.tau0_vec(x2, x3, tau); // pre-stress [dip, strike] [Pa]
    bp5_params_.V_init_vec(x2, x3, Vi); // initial velocity [dip, strike] [m/s]
}

```

7.3 FaultBasis

File: fault/fault_basis.hpp

Created during `SetupFaultInfo()` in the domain operator (Section 5.2). Used by `ComputeTraction()` to rotate traction to fault-local coordinates, and by `AssembleSlipContribution()` to embed slip into global 3D.

Computes per-face local coordinate frames (normal, dip, strike) from the mesh geometry.

Algorithm (`ComputeOrientedFrame`, line 366):

Given a raw face normal $\mathbf{n}_{\text{raw}}$ from the mesh:

1. Record the raw normal length: $n_l = |\mathbf{n}_{\text{raw}}|$
2. Check orientation against a reference normal $\mathbf{n}_{\text{ref}}$ (default: $(0, -1, 0)$ for $Y = 0$ faults):
 - If $\mathbf{n}_{\text{raw}} \cdot \mathbf{n}_{\text{ref}} < 0$: the normal points away from the reference. Set `sign_flipped = true` and reverse $\mathbf{n}_{\text{raw}}$.
3. Normalize: $\hat{\mathbf{n}} = \mathbf{n}_{\text{raw}}/n_l$
4. Compute strike direction: $\hat{\mathbf{s}} = \text{normalize}(\mathbf{up} \times \hat{\mathbf{n}})$ where $\mathbf{up} = (0, 0, -1)$
5. Compute dip direction: $\hat{\mathbf{d}} = \hat{\mathbf{s}} \times \hat{\mathbf{n}}$ (unit since $\hat{\mathbf{s}} \perp \hat{\mathbf{n}}$)
6. **If `sign_flipped`: negate all three vectors ($\hat{\mathbf{n}}, \hat{\mathbf{d}}, \hat{\mathbf{s}}$)**

Why negate when `sign_flipped`? In a DG mesh, the two elements sharing a face see opposite face normals. The element whose normal opposes $\mathbf{n}_{\text{ref}}$ gets `sign_flipped = true`. By negating all stored vectors, the embedded slip $\delta = s_{\text{dip}}\hat{\mathbf{d}} + s_{\text{strike}}\hat{\mathbf{s}}$ has opposite sign on the two sides, which is exactly the DG jump: the displacement discontinuity $[[\mathbf{u}]] = \mathbf{u}^- - \mathbf{u}^+$ requires opposite-sign contributions from each element. The sign is **baked into the stored vectors** so that callers (slip embedding, traction projection) can use them directly without any additional sign logic.

Key operations:

- `ProjectTraction(fi, traction_global, tau_local)`: $\tau_{\text{dip}} = \mathbf{T} \cdot \hat{\mathbf{d}}, \tau_{\text{strike}} = \mathbf{T} \cdot \hat{\mathbf{s}}$
- `EmbedSlip(fi, slip_local, delta_u_global)`: $\delta = s_{\text{dip}}\hat{\mathbf{d}} + s_{\text{strike}}\hat{\mathbf{s}}$
- `EmbedSlipQP(fi, q, slip_local, delta_u)`: same but using per-quadrature-point tangent vectors (for curved faces or multi-DOF)
- `NormalStress(fi, traction_global)`: $\sigma_n = -\mathbf{T} \cdot \hat{\mathbf{n}}$ (positive = compression)

7.4 FaceQuadrature and L2 Projection

File: `fault/face_quadrature.hpp`

Created during `SetupFaultInfo()`. Used by traction recovery (Section 6.2.4) and slip RHS assembly when polynomial order $p \geq 2$ (multi-DOF per fault face).

Provides basis functions and L2 projection for the fault surface discretization:

- At $p = 0$: `nbf = 1` (constant per face, equivalent to face averaging)
- At $p \geq 1$: `nbf = (p+1)(p+2)/2` DOFs per triangular face

Why L2 projection is needed After `ComputeTractionAtQuadPoints()` (Section 6.2.4) computes traction values $T(x_q)$ at face quadrature points in global 3D coordinates, we need to convert them to per-DOF values in fault-local coordinates for the friction solver. When `nbf > 1` (multi-DOF), simply evaluating at DOF nodes is inaccurate because the DG face integral was computed at quadrature points that differ from the DOF node positions. Instead, we perform a **Galerkin L2 projection**: find the polynomial T_h on the face that is closest to $T(x_q)$ in the L^2 sense.

Derivation We want coefficients τ_k ($k = 1, \dots, \text{nbf}$) such that the face polynomial $T_h(x) = \sum_k \tau_k \phi_k(x)$ minimizes $\|T - T_h\|_{L^2(e)}^2$. The optimality condition is:

$$\int_e T_h \phi_l |J_e| d\hat{s} = \int_e T \phi_l |J_e| d\hat{s}, \quad l = 1, \dots, \text{nbf}$$

Substituting $T_h = \sum_k \tau_k \phi_k$ and using quadrature:

$$\sum_k \underbrace{\left(\sum_q w_q |J_e(x_q)| \phi_k(x_q) \phi_l(x_q) \right)}_{M_{\text{phys}}[l,k]} \tau_k = \underbrace{\sum_q w_q |J_e(x_q)| \phi_l(x_q) T(x_q)}_{b_l}$$

where $|J_e|$ is the face Jacobian determinant. For **flat faces** (planar fault), $|J_e|$ is constant and cancels: $M_{\text{phys}} = |J_e| \cdot M_{\text{ref}}$ and $b = |J_e| \cdot b_{\text{ref}}$, so:

$$\tau = M_{\text{ref}}^{-1} \mathbf{b}_{\text{ref}}, \quad \text{where} \quad M_{\text{ref}}[l, k] = \sum_q w_q \phi_l(x_q) \phi_k(x_q), \quad b_{\text{ref}}[l] = \sum_q w_q \phi_l(x_q) T(x_q)$$

M_{ref}^{-1} is precomputed once in the `FaceQuadrature` constructor and stored as `M_ref_inv_`.

Code implementation

GalerkinProject (face_quadrature.hpp, line 216):

```
void GalerkinProject(int ncomp, const Vector &quad_vals, Vector &nodal_vals) const
{
    // Step 1: Compute RHS  b_ref[l] = sum_q w_q * phi_l(q) * T(q)
    Vector rhs(ncomp * nbf_);
    rhs = 0.0;
    for (int c = 0; c < ncomp; c++)
        for (int l = 0; l < nbf_; l++)
        {
            real_t val = 0.0;
            for (int q = 0; q < nq_; q++)
                val += ir_.IntPoint(q).weight * e_q_(l, q) * quad_vals(c * nq_ + q);
            rhs(c * nbf_ + l) = val;
        }

    // Step 2: Apply M_ref_inv:  tau = M_ref_inv * b_ref
    nodal_vals.SetSize(ncomp * nbf_);
    for (int c = 0; c < ncomp; c++)
        for (int k = 0; k < nbf_; k++)
        {
            real_t val = 0.0;
            for (int l = 0; l < nbf_; l++)
                val += M_ref_inv_(k, l) * rhs(c * nbf_ + l);
            nodal_vals(c * nbf_ + k) = val;
        }
}
```

```

    }
}

```

Here $\mathbf{e}_q(\mathbf{l}, \mathbf{q}) = \phi_l(x_q)$ is the basis function l evaluated at quadrature point q , `ir_.IntPoint(q).weight` = w_q , and $\mathbf{M}_{\text{ref_inv}}(\mathbf{k}, \mathbf{l}) = [M_{\text{ref}}^{-1}]_{kl}$. At `nbf = 1`, this reduces to: $\tau_0 = (\sum_q w_q T_q) / (\sum_q w_q) =$ face average.

8. Friction and State Evolution

Consumed by Step 4 of each ODE evaluation: `ComputeRHS()` calls the friction solver to determine slip rate from stress, then the state evolution law to compute $\dot{\psi}$.

8.1 DieterichRuinaFriction

File: `friction/dieterich_ruina.hpp`

Implements the regularized Dieterich-Ruina rate-and-state friction law.

Friction coefficient in ψ -space (line 278):

$$f(V, \psi) = a \cdot \operatorname{asinh} \left[\frac{V}{2V_0} \exp \left(\frac{\psi}{a} \right) \right]$$

where V = slip rate [m/s], ψ = logarithmic state variable [-], a = rate-and-state direct effect parameter [-], V_0 = reference slip rate [m/s].

Friction coefficient in psi-space (`dieterich_ruina.hpp`, line 278):

```

real_t FrictionCoefficientPsi(real_t V, real_t psi, real_t a) const {
    if (V <= 0.0) { return 0.0; }
    if (psi / a > 700.0) { // overflow guard: asinh(x) ~ log(2x) for large x
        return std::max(0.0, a * std::log(V / cp_.V0) + psi);
    }
    real_t sinh_arg = (V / (2.0 * cp_.V0)) * std::exp(psi / a);
    return a * std::asinh(sinh_arg);
}

```

Scalar slip rate solver – Brent’s method in $\log_{10}(V)$ space (line 322). Solves the friction equation for V given total stress τ , state ψ , normal stress σ_n , radiation damping η , and direct effect a :

$$\tau = \sigma_n f(V, \psi) + \eta V$$

The residual function is $F(V) = \tau - \sigma_n f(V, \psi) - \eta V$, which is monotonically decreasing in V (guaranteeing a unique root). Working in $\log_{10}(V)$ space improves convergence across the 30+ orders of magnitude that V spans (from 10^{-30} m/s interseismic to 10^0 m/s coseismic):

Brent solver in log10(V) space (`dieterich_ruina.hpp`, line 322):

```

auto fF = [&](real_t Ve) -> real_t { // Ve = log10(V)
    real_t V = std::pow(10.0, Ve);

```

```

    return tau - sigma_n * FrictionCoefficientPsi(V, psi, a) - eta * V;
};
// Initial bracket: [Va = -32, Vb = log10(tau/eta)]
// Fallback bracket: [Va_min = -300, Vb] (handles extreme psi values)
real_t Ve = zeroIn(lo, hi, fF); // Brent's method [line 559]
return std::pow(10.0, Ve);

```

Vector slip rate solver (line 470) – for 3D (two-component stress/slip rate):

1. Compute scalar stress magnitude: $|\tau| = \sqrt{\tau_{\text{dip}}^2 + \tau_{\text{strike}}^2}$
2. Solve scalar friction equation for $|V|$ using Brent's method above
3. Decompose: $\mathbf{V} = -(|V|/|\tau|) \tau$ (slip rate anti-parallel to stress)

Vector slip rate decomposition (dieterich_ruina.hpp, line 470):

```

real_t tau_abs = sqrt(tau_vec[0]*tau_vec[0] + tau_vec[1]*tau_vec[1]);
real_t V_abs = SolveSlipRatePsi(tau_abs, psi, sigma_n, eta, a, ...);
real_t inv_tau = 1.0 / tau_abs;
V_vec[0] = -(tau_vec[0] * inv_tau) * V_abs; // anti-parallel to tau
V_vec[1] = -(tau_vec[1] * inv_tau) * V_abs;

```

Initial psi from equilibrium (line 506): Given pre-stress τ_0 and initial slip rate V_{init} , invert the friction law: $\psi_0 = a \ln \left[\frac{2V_0}{V_{\text{init}}} \sinh \left(\frac{\tau_0 - \eta V_{\text{init}}}{a\sigma_n} \right) \right]$

8.2 State Evolution Laws

File: friction/state_evolution.hpp

All implement `Rate(V, psi_or_theta, Dc)` returning the time derivative of the state variable:

Class	Equation	Used by
AgingLawPsi	$\dot{\psi} = \frac{bV_0}{D_c} \left[\exp\left(\frac{f_0 - \psi}{b}\right) - \frac{V}{V_0} \right]$	BP5 (default)
AgingLaw	$\dot{\theta} = 1 - V\theta/D_c$	BP2 (theta-space)
SlipLawPsi	$\dot{\psi} = -(V/D_c)(\psi - \psi_{ss})$ where $\psi_{ss} = f_0 + b \ln(V_0/V)$	Alternative
SlipLaw	$\dot{\theta} = -(V\theta/D_c) \ln(V\theta/D_c)$	Alternative

AgingLawPsi implementation (state_evolution.hpp, line 190):

```

// AgingLawPsi::Rate [line 190]:
real_t Rate(real_t V, real_t psi, real_t Dc) const override {
    real_t exp_arg = (f0_ - psi) / b_;
    return (b_ * V0_ / Dc) * (std::exp(exp_arg) - V / V0_);
}
// No cap on exp_arg -- matches the SCEC benchmark formulation exactly.
// The adaptive RK45 stepper handles the stiffness by reducing dt.

```

9. Time Integration

Configured at Stage 7 of the driver. Drives the ODE evaluation loop (Section 2) throughout Stage 8.

9.1 PETSc TS (default)

File: drivers/seas_driver.cpp (lines 458-506)

The default time stepper uses PETSc's TS framework with the rk5dp method (Dormand-Prince 5(4)) and TSAdaptBasic adaptive step size control. Configuration is via a PETSc options file.

PETSc TS setup (seas_driver.cpp, lines 458-506):

```
petsc_ode = std::make_unique<PetscODESolver>(mpi.GetComm(), "");
petsc_ode->Init(seas_op, PetscODESolver::ODE_SOLVER_GENERAL);
// Re-enable adaptive stepping (MFEM disables it by default)
TSGetAdapt(ts, &tsad);
TSAdaptSetType(tsad, TSADAPTBASIC);
TSSetFromOptions(ts); // apply options from .cfg file
TSSetTimeStep(ts, dt_init);
// Register I/O callback: called after every accepted step
TSMonitorSet(ts, driver_ts_monitor_callback, &petsc_mon_ctx, nullptr);
// Run entire simulation in one call:
petsc_ode->Run(state, t, current_dt, t_final);
```

Default tolerances: atol = 1e-7, rtol = 1e-50 (pure absolute tolerance).

9.2 Native DormandPrince RK45 (fallback)

File: solver/time_stepper.hpp (line 130)

Used when PETSc is not available (use_petsc_ts = false). Implements the same DOPRI5(4) algorithm: 7 stages with FSAL, 5th-order solution advances state, 4th-order gives error estimate.

Error norm (weighted L^∞ , lines 490-508):

Weighted error norm computation:

```
for (int i = 0; i < n; i++) {
    real_t scale = atol_ + rtol_ * std::abs(y_tmp(i)); // per-DOF tolerance
    real_t ei = std::abs(err_(i)) / scale; // normalized error
    err_norm = max(err_norm, ei);
}
if (mpi_ctx_) { err_norm = mpi_ctx_->GlobalMax(err_norm); } // consistent across ranks
```

Step size control (PI controller, line 532):

$$\Delta t_{\text{new}} = \text{safety} \cdot \Delta t \cdot \|\mathbf{e}\|^{-1/5}$$

Step size PI controller (line 532):

```
dt_new = safety_ * dt * std::pow(err_norm, -0.2);
dt_new = clamp(dt_new, shrink_min_ * dt, growth_max_ * dt);
dt_new = clamp(dt_new, dt_min_, dt_max_);
```

Accept if $\|\mathbf{e}\| \leq 1$; reject otherwise (apply extra `reject_safety = 0.5` shrink).

V-guard (line 252): Safety feature that rejects a step if the maximum slip rate in any RK stage exceeds `v_guard_factor * v_max_stage0`. This catches CFL-violating steps before they produce NaN.

10. Constitutive Models

Consumed by the domain operator (Section 5.2) for stress evaluation and DG penalty computation.

10.1 ConstitutiveModel (abstract)

File: `constitutive/constitutive_model.hpp`

Three-tier design for extensibility: - **Tier 1 (Algebraic)**: Linear elastic – no internal state, constant tangent - **Tier 2 (Evolving)**: e.g., damage-breakage – state ODEs integrated alongside fault - **Tier 3 (Non-local)**: e.g., gradient-regularized damage – state + FE gradient terms

Currently only Tier 1 is implemented. Key methods: - `ComputeStress(epsilon, int_vars, sigma)`: strain $\rightarrow$ stress - `ComputeTangent(epsilon, int_vars, C_tang)`: 6x6 Voigt tangent stiffness - `GetPenaltyModulus()`: max eigenvalue of C ($= \lambda + 2\mu$), used for DG penalty - `GetMaxWaveSpeed(rho)`: $\sqrt{(\lambda + 2\mu)/\rho}$, used for CFL and radiation damping

10.2 LinearElastic

File: `constitutive/linear_elastic.hpp`

Isotropic Hooke's law: $\sigma = \lambda \text{tr}(\varepsilon) \mathbf{I} + 2\mu \varepsilon$

Voigt notation: $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$ where $\gamma_{ij} = 2\varepsilon_{ij}$ (engineering strain).

Stress computation (`linear_elastic.hpp`, `ComputeStress`):

```
void ComputeStress(const real_t *epsilon, const real_t *int_vars, real_t *sigma) const {
    real_t tr_eps = epsilon[0] + epsilon[1] + epsilon[2];
    sigma[0] = lambda_ * tr_eps + 2.0 * mu_ * epsilon[0]; // sigma_xx
    sigma[1] = lambda_ * tr_eps + 2.0 * mu_ * epsilon[1]; // sigma_yy
    sigma[2] = lambda_ * tr_eps + 2.0 * mu_ * epsilon[2]; // sigma_zz
    sigma[3] = mu_ * epsilon[3]; // tau_xy = mu * gamma_xy (NOT 2*mu)
    sigma[4] = mu_ * epsilon[4]; // tau_yz
    sigma[5] = mu_ * epsilon[5]; // tau_xz
}
```

Tangent stiffness: $C_{00} = C_{11} = C_{22} = \lambda + 2\mu$, $C_{01} = C_{02} = C_{12} = \lambda$, $C_{33} = C_{44} = C_{55} = \mu$ (engineering Voigt: shear diagonal is μ , not 2μ).

11. Parallelism and MPI

The simulation runs in parallel using MPI. Each rank owns a subset of mesh elements (via MFEM's `ParMesh`), a portion of fault faces, and the corresponding fault DOFs. MPI communication is

required at specific points in the ODE evaluation loop.

11.1 MPI Operations in the ODE Evaluation Flow

The following flowchart shows where MPI operations occur within a single `Mult(state, rate)` call (Section 2):

```

Mult(state, rate)
|
| Step 1: GetSlip(state, slip_)          -- LOCAL only, no MPI
|
| Step 2a: ExpandOwnedToLocalFault()
|   +--- FaultScatter::BeginScatter()    -- MPI_Isend + MPI_Irecv (point-to-point)
|   +--- FaultScatter::WaitScatter()     -- MPI_Waitall
|   (ghost fault DOFs now consistent across ranks)
|
| Step 2b: Solve(t, local_slip_, u_gf_)
|   +--- AssembleStiffness() [first call only]
|   |   +--- ParMesh::ExchangeFaceNbrData() -- MPI (neighbor data exchange)
|   |   +--- ParBilinearForm::Assemble()    -- MPI (parallel matrix assembly)
|   +--- AssembleSlipContribution()        -- LOCAL (uses local fault view)
|   +--- AssembleSlipContributionShared()  -- LOCAL (shared faces, Elem1 only)
|   +--- AssembleDirichletLoading()        -- LOCAL (boundary faces)
|   +--- HypreParMatrix * solver_ -> Mult(B,X) -- MPI (parallel sparse solve: MUMPS/CG)
|
| Step 3: ComputeTraction(u_gf_, local_slip_, local_traction_)
|   +--- Interior fault faces              -- LOCAL
|   +--- Shared fault faces               -- LOCAL (uses face neighbor data)
|   +--- RestrictToOwnedFault()           -- LOCAL (extract owned subset)
|
| Step 4: ComputeRHS(traction_, state, rate) -- LOCAL (per-DOF friction solve)
|
| After Mult returns to the time stepper:
|   +--- DormandPrinceRK45: error norm
|   |   +--- MPIContext::GlobalMax(err_norm) -- MPI_Allreduce(MAX)
|   |   (ensures consistent accept/reject across all ranks)
|   +--- V_max reporting
|       +--- MPIContext::GlobalMax(V_max)    -- MPI_Allreduce(MAX)

```

Critical MPI safety rule: The error norm reduction in the time stepper (`GlobalMax(err_norm)`) ensures all ranks make the same accept/reject decision. Without this, ranks can diverge: one rank accepts while another rejects, causing deadlock in subsequent MPI collectives inside `Solve()`.

11.2 MPIContext

File: `common/mpi_context.hpp`

RAII wrapper for MPI initialization. All collective operations use a stored communicator. In serial mode (no `SEAS_USE_MPI`), all methods are no-ops returning the local value.

Method	MPI Call	Used in
GlobalMax(val)	MPI_Allreduce(MAX)	Error norm (time stepper), V_max, equilibrium error
GlobalSum(val)	MPI_Allreduce(SUM)	RMS error norm (optional 2-norm mode)
GlobalSumInt(val)	MPI_Allreduce(SUM)	Global fault DOF count, NaN detection
GatherToRoot(local, global)	MPI_Gatherv	I/O: collect fault data on rank 0 for file output
Barrier()	MPI_Barrier	File cleanup before I/O setup
Bcast(vec)	MPI_Bcast	Broadcast global data from root

11.3 FaultScatter

File: common/fault_scatter.hpp

Used by ElasticityDomainOperator::ExpandOwnedToLocalFault() at Step 2a of every ODE evaluation.

In parallel DG, a fault face at a partition boundary is a “shared face” that appears on both ranks. Each rank owns one copy (the “owned” DOFs) and needs a ghost copy from the neighbor. FaultScatter handles this exchange using non-blocking MPI:

FaultScatter usage (inside ExpandOwnedToLocalFault):

```
// 1. Post non-blocking receives and sends for all neighbor ranks
scatter_>BeginScatter(owned_data, comps_per_dof, nbf_per_face);
// Internally: MPI_Irecv from each neighbor, then pack send buffer
// and MPI_Isend to each neighbor

// 2. Wait for all transfers to complete
scatter_>WaitScatter(); // MPI_Waitall

// 3. Unpack received ghost data into local_data at ghost face positions
for (int bi = 0; bi < scatter_>NumBlocks(); bi++) {
    const Vector &recv = scatter_>GetRecvBuffer(bi);
    // ... copy recv data into local_data at ghost DOF indices
}
```

The communication pattern is established once during SetupFaultInfo() by identifying which fault faces are shared between ranks and building SharedFaultCommBlock structures that record which owned faces to send and which ghost faces to receive.

12. I/O and Benchmark Output

Configured at Stage 6 of the driver. Called during Stage 8 (time loop) after each accepted step.

12.1 ProbeOutput

File: io/probe_output.hpp

Single-station ASCII file writer for time-series data:

```
ProbeOutput(filename, {"time(s)", "log10(Vmax)(m/s)"}, "BP5-QD global output");
global_out->WriteStep({t, std::log10(V_max)});
```

12.2 ParallelBP5BenchmarkOutput

File: io/bp5_parallel_output.hpp

Distributed SCEC fltst format output. Each station is identified by $(x_{\text{strike}}, z_{\text{depth}})$ coordinates. Uses `FaultGeometry::GatherToRoot()` and `FaceQuadrature::GalerkinProject()` to assemble fault-wide data from distributed DOFs. Implements an adaptive write schedule: more frequent during seismic events (high V_{max}), less frequent interseismically.

13. Coordinate Conventions and Sign Rules

Coordinate System

SCEC	Code/Mesh	Direction
x_1 (fault-normal)	Y	fault at Y=0
x_2 (along-strike)	X	
x_3 (depth, +down)	-Z	Z=0 surface, Z<0 depth

Mesh Boundary Attributes

Tag	BC Type	Location
1	Natural	Top (Z=0) + Bottom
3	Fault	Y=0 interior faces
5	Dirichlet	Far-field (Y=+/-Y1, X=+/-X1)

Sign Rules

- **Slip rate direction:** Anti-parallel to traction: $\mathbf{V} = -(|V|/|\tau|)\tau$
- **Normal stress:** $\sigma_n > 0$ = compression (geology convention)
- **Elastic sigma_n feedback:** $\sigma_{n,\text{eff}} = \sigma_{n,\text{preset}} + T_n^{\text{elastic}}$ where $T_n = -\mathbf{T} \cdot \hat{\mathbf{n}}$ (positive in compression)
- **FaultBasis vectors:** Sign already includes the orientation correction (see Section 7.3 for explanation)
- **DG face sign in assembly:** $\text{sign} = (\text{nor}(1) > 0) ? 1.0 : -1.0$ (Y-component of face normal)

14. State Vector Layout

Antiplane (**SlipComponents=1, StatePerNode=2**)

state = [slip_0, psi_0, slip_1, psi_1, ..., slip_{N-1}, psi_{N-1}]

- state(2*i + 0) = slip at node i [m]
- state(2*i + 1) = state variable (θ or ψ)
- rate(2*i + 0) = slip rate V [m/s]
- rate(2*i + 1) = $\dot{\theta}$ or $\dot{\psi}$

3D Elasticity (**SlipComponents=2, StatePerNode=3**)

state = [s_dip_0, s_strike_0, psi_0, s_dip_1, s_strike_1, psi_1, ...]

- state(3*i + 0) = dip slip [m]
- state(3*i + 1) = strike slip [m]
- state(3*i + 2) = ψ (logarithmic state variable) [-]
- rate(3*i + 0) = V_{dip} [m/s]
- rate(3*i + 1) = V_{strike} [m/s]
- rate(3*i + 2) = $\dot{\psi}$

Slip and Traction Vectors

- **Slip** [SlipComponents * N]: interleaved [dip_0, strike_0, dip_1, strike_1, ...]
 - **Traction** [SlipComponents * N]: same layout as slip
 - **Normal traction** [N]: one scalar per node ($\sigma_n > 0$ = compression)
-

15. Appendix: DG-IP Derivation for 3D Elasticity

This appendix derives the Interior Penalty DG formulation implemented in Section 6, starting from the strong form PDE and arriving at the discrete bilinear form that maps directly to `AssembleFaceMatrix()`. Every step is shown.

15.1 Strong Form

The quasi-static momentum balance for linear elasticity in domain Ω with boundary $\partial\Omega = \Gamma^D \cup \Gamma^N \cup \Gamma^F$:

$$-\nabla \cdot \sigma = \mathbf{b} \quad \text{in } \Omega$$

$$\sigma \mathbf{n} = \mathbf{T} \quad \text{on } \Gamma^N$$

$$\mathbf{u} = \mathbf{g}^D \quad \text{on } \Gamma^D$$

$$\mathbf{R}[\mathbf{u}] = \delta \quad \text{on } \Gamma^F$$

where $\sigma = \mathbb{C} : \varepsilon$, $\varepsilon = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$, $C_{ijkl} = \lambda \delta_{ij} \delta_{kl} + \mu(\delta_{ik} \delta_{jl} + \delta_{il} \delta_{jk})$, $\mathbf{R}$ is the global-to-fault rotation matrix, δ is the prescribed slip, and $\mathbf{b}$ is the body force (zero for quasi-static SEAS).

15.2 First-Order System and Weak Form

Rewrite as a first-order system $(\mathbf{u}, \sigma)$ by dropping the index notation:

$$\sigma = \mathbb{C} : \nabla \mathbf{u}$$

$$-\nabla \cdot \sigma = \mathbf{b} \quad \text{in } \Omega$$

Multiply the first equation by a tensor test function τ and the second by a vector test function $\mathbf{v}$, integrate over a single element K :

$$\int_K \tau : \sigma \, dx = \int_K \tau : \mathbb{C} : \nabla \mathbf{u} \, dx$$

$$\int_K \nabla \mathbf{v} : \sigma \, dx - \int_{\partial K} \mathbf{v} \cdot \sigma \mathbf{n}_K \, ds = \int_K \mathbf{v} \cdot \mathbf{b} \, dx$$

where the second equation uses integration by parts: $\int_K \mathbf{v} \cdot (-\nabla \cdot \sigma) = \int_K \nabla \mathbf{v} : \sigma - \int_{\partial K} \mathbf{v} \cdot \sigma \mathbf{n}_K$.

15.3 Flux Form

In DG, $\mathbf{u}$ and σ are double-valued on element boundaries ∂K . Replace the traces with numerical fluxes $\hat{\mathbf{u}}$ and $\hat{\sigma}$:

$$\int_K \tau : \sigma \, dx + \int_K (\nabla \cdot (\tau : \mathbb{C})^T) \cdot \mathbf{u} \, dx - \int_{\partial K} \hat{\mathbf{u}} \cdot (\tau : \mathbb{C}) \mathbf{n}_K \, ds = 0$$

$$\int_K \nabla \mathbf{v} : \sigma \, dx - \int_{\partial K} \mathbf{v} \cdot \hat{\sigma} \mathbf{n}_K \, ds = \int_K \mathbf{v} \cdot \mathbf{b} \, dx$$

The first equation is obtained by applying the product rule identity $\tau : \mathbb{C} : \nabla \mathbf{u} = \nabla \cdot [(\tau : \mathbb{C})^T \mathbf{u}] - (\nabla \cdot (\tau : \mathbb{C})^T) \cdot \mathbf{u}$ and then replacing boundary traces of $\mathbf{u}$ with $\hat{\mathbf{u}}$.

15.4 Eliminating σ (Primal Form)

Goal: eliminate σ to obtain a bilinear form in $\mathbf{u}$ and $\mathbf{v}$ only.

Step 1. From the first flux-form equation, set $\tau = \nabla \mathbf{v}$ (valid since $\nabla_h V_h \subset \Sigma_h$):

$$\int_K \nabla \mathbf{v} : \sigma \, dx = \int_K \nabla \mathbf{v} : \mathbb{C} : \nabla \mathbf{u} \, dx + \int_{\partial K} (\hat{\mathbf{u}} - \mathbf{u}) \cdot (\nabla \mathbf{v} : \mathbb{C}) \mathbf{n}_K \, ds$$

Step 2. Substitute into the second flux-form equation:

$$\int_K \nabla \mathbf{v} : \mathbb{C} : \nabla \mathbf{u} \, dx + \int_{\partial K} (\hat{\mathbf{u}} - \mathbf{u}) \cdot (\nabla \mathbf{v} : \mathbb{C}) \mathbf{n}_K \, ds - \int_{\partial K} \mathbf{v} \cdot \hat{\boldsymbol{\sigma}} \mathbf{n}_K \, ds = \int_K \mathbf{v} \cdot \mathbf{b} \, dx$$

This is a single equation in $(\mathbf{u}, \mathbf{v})$ with numerical fluxes $\hat{\mathbf{u}}$ and $\hat{\boldsymbol{\sigma}}$ on faces.

15.5 Summing Over Elements and the Magic Identity

Sum over all elements $K \in \mathcal{T}_h$. The boundary integrals $\sum_K \int_{\partial K}$ become sums over faces. On an interior face e shared by K^- and K^+ (with $\mathbf{n}$ from K^- to K^+), the following identity converts element-boundary sums to jump/average notation:

$$\sum_K \int_{\partial K} q_K \varphi_K \cdot \mathbf{n}_K \, ds = \int_{\Gamma} \llbracket q \rrbracket \cdot \{\{\varphi\}\} \, ds + \int_{\Gamma^0} \{\{q\}\} \llbracket \varphi \rrbracket \, ds$$

15.6 Choosing the IP Numerical Fluxes

For the **Interior Penalty** method on an interior face $e \in \Gamma^0 \cup \Gamma^F$:

$$\hat{\mathbf{u}} = \{\{\mathbf{u}\}\}, \quad \hat{\boldsymbol{\sigma}} \mathbf{n} = \{\{\mathbb{C} : \nabla \mathbf{u}\}\} \mathbf{n} - \frac{\eta_e}{h_e} \llbracket \mathbf{u} \rrbracket$$

On a **fault face** $e \in \Gamma^F$, the displacement flux is double-valued to enforce the slip:

$$\hat{\mathbf{u}}|_{K^-} = \{\{\mathbf{u}\}\} + \frac{1}{2} \delta, \quad \hat{\mathbf{u}}|_{K^+} = \{\{\mathbf{u}\}\} - \frac{1}{2} \delta$$

so that $\llbracket \hat{\mathbf{u}} \rrbracket = \delta$ (the prescribed slip). The stress flux includes the same correction:

$$\hat{\boldsymbol{\sigma}} \mathbf{n} = \{\{\mathbb{C} : \nabla \mathbf{u}\}\} \mathbf{n} - \frac{\eta_e}{h_e} (\llbracket \mathbf{u} \rrbracket - \delta)$$

On a **Dirichlet boundary** $e \in \Gamma^D$: $\hat{\mathbf{u}} = \mathbf{g}^D$, $\hat{\boldsymbol{\sigma}} \mathbf{n} = (\mathbb{C} : \nabla \mathbf{u}) \mathbf{n} - \frac{\eta_e}{h_e} (\mathbf{u} - \mathbf{g}^D)$.

On a **Neumann boundary** $e \in \Gamma^N$: $\hat{\boldsymbol{\sigma}} \mathbf{n} = \mathbf{0}$, $\hat{\mathbf{u}} = \mathbf{u}$ (no contribution).

15.7 Assembling the Primal Form

Substituting the IP fluxes into the summed primal equation (Step 2 of Section 15.4) and separating terms that depend on both $\mathbf{u}$ and $\mathbf{v}$ (bilinear form a) from terms that depend only on $\mathbf{v}$ (linear form L), we obtain:

Bilinear form (terms I, II-a, III-a, III-b from the derivation):

$$a(\mathbf{u}, \mathbf{v}) = \underbrace{\sum_K \int_K \nabla \mathbf{v} : \mathbb{C} : \nabla \mathbf{u} \, dx}_{\text{(I) volume}}$$

$$\begin{aligned}
& - \underbrace{\sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e ([\mathbf{u}] \cdot \{\{\mathbb{C} : \nabla \mathbf{v}\}\} \mathbf{n} + [\mathbf{v}] \cdot \{\{\mathbb{C} : \nabla \mathbf{u}\}\} \mathbf{n}) ds}_{\text{(II-a + III-a) consistency + symmetry on interior/fault faces}} \\
& + \underbrace{\sum_{e \in \Gamma^0 \cup \Gamma^F} \int_e \frac{\eta_e}{h_e} [\mathbf{u}] \cdot [\mathbf{v}] ds}_{\text{(III-b) interior/fault penalty}} \\
& - \underbrace{\sum_{e \in \Gamma^D} \int_e (\mathbf{u} \cdot (\mathbb{C} : \nabla \mathbf{v}) \mathbf{n} + \mathbf{v} \cdot (\mathbb{C} : \nabla \mathbf{u}) \mathbf{n}) ds}_{\text{Dirichlet consistency + symmetry}} \\
& + \underbrace{\sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} \mathbf{u} \cdot \mathbf{v} ds}_{\text{Dirichlet penalty}}
\end{aligned}$$

Linear form (terms from Dirichlet data $\mathbf{g}^D$ and fault slip δ):

$$\begin{aligned}
L(\mathbf{v}) = & - \underbrace{\sum_{e \in \Gamma^D} \int_e \mathbf{g}^D \cdot (\mathbb{C} : \nabla \mathbf{v}) \mathbf{n} ds}_{\text{Dirichlet data}} + \sum_{e \in \Gamma^D} \int_e \frac{\eta_e}{h_e} \mathbf{g}^D \cdot \mathbf{v} ds \\
& - \underbrace{\sum_{e \in \Gamma^F} \int_e \delta \cdot \{\{\mathbb{C} : \nabla \mathbf{v}\}\} \mathbf{n} ds + \sum_{e \in \Gamma^F} \int_e \frac{\eta_e}{h_e} \delta \cdot [\mathbf{v}] ds}_{\text{fault slip data}}
\end{aligned}$$

15.8 Structure of the Bilinear Form

The bilinear form $a(\mathbf{u}, \mathbf{v})$ has three types of terms:

1. **Volume term** (I): positive, standard continuous Galerkin stiffness. Implemented by `ElasticityIntegrator`.
2. **Consistency + symmetry** (II-a + III-a): negative, couple test and trial across the face. These terms ensure that the exact solution satisfies the discrete equations (consistency) and that $a(\cdot, \cdot)$ is symmetric (symmetry). Implemented by `AssembleFaceBlock` with coefficients `c0` (consistency) and `c1` (symmetry).
3. **Penalty** (III-b): positive, penalizes the jump $[\mathbf{u}]$ to enforce inter-element continuity. The penalty parameter η_e/h_e must be sufficiently large for coercivity. Implemented by `AssembleFaceBlock` with coefficient `c2`.

15.9 Mapping to Code

The bilinear form terms map to `AssembleFaceBlock` as follows (for an interior face, K^- = element 1, K^+ = element 2):

Weak form term	Block (K^-, K^-)	Block (K^-, K^+)	Block (K^+, K^-)	Block (K^+, K^+)
Consistency (c0)	$-\frac{1}{2}$	$-\frac{1}{2}$	$+\frac{1}{2}$	$+\frac{1}{2}$
Symmetry (c1, $\varepsilon = -1$)	$-\frac{1}{2}$	$+\frac{1}{2}$	$-\frac{1}{2}$	$+\frac{1}{2}$
Penalty (c2)	$+\alpha$	$-\alpha$	$-\alpha$	$+\alpha$

The sign pattern arises because: on element 1, $\llbracket v \rrbracket_1 = +v$ and $\llbracket u \rrbracket_1 = +u$; on element 2, $\llbracket v \rrbracket_2 = -v$ and $\llbracket u \rrbracket_2 = -u$. The penalty is positive on same-element blocks and negative on cross-element blocks.

The linear form maps to **AssembleSlipFaceRHS** (fault slip δ) and **AssembleBoundaryFaceRHS** (Dirichlet $\mathbf{g}^D$). Each uses the same traction operator and penalty computation as the bilinear form, ensuring K-b consistency.